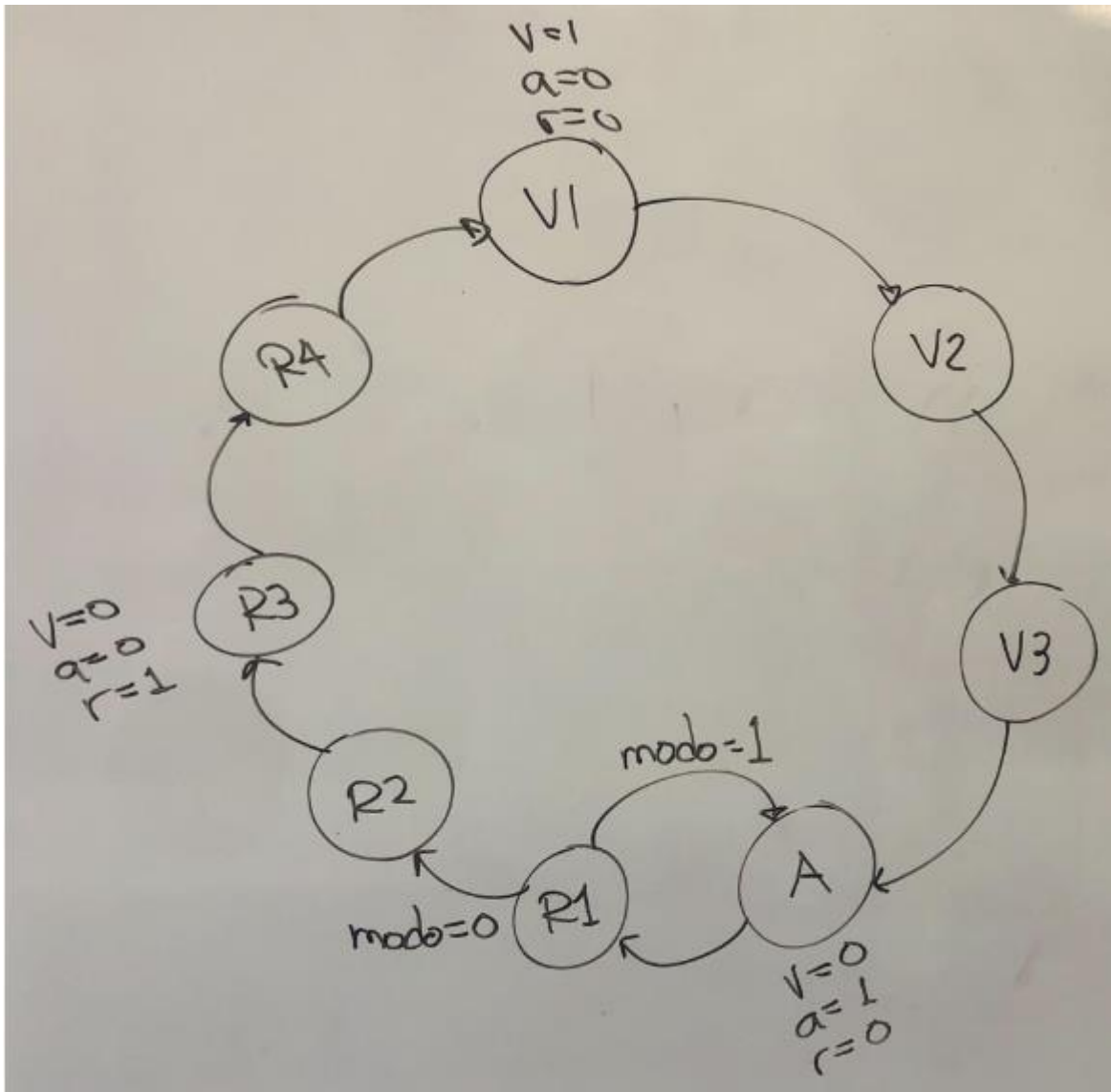


Joshua Alexis Máynez Alcayde
243990

Practice #7: States machines with Verilog in Cadence

Describe with Verilog the following state machine.



HDL Code

```
1 //Verilog HDL for "Verilog", "FSM" "functional"
2 module FSM (V_out, A_out, R_out, Clk, Modo);
3
4 input Clk;
5 input Modo;
6 output V_out, A_out, R_out;
7 reg VS_out = 0;
8 reg AS_out = 1;
9 reg RS_out = 0;
10 reg [2:0] state = 0;
11 reg [2:0] nextstate = 3'b001;
12
13 localparam A = 3'b000;
14 localparam R1 = 3'b001;
15 localparam R2 = 3'b010;
16 localparam R3 = 3'b011;
17 localparam R4 = 3'b100;
18 localparam V1 = 3'b101;
19 localparam V2 = 3'b110;
20 localparam V3 = 3'b111;
21
22 //state register
23 always @(posedge Clk) begin
24     if (Modo == 1'b1 && state == R1)begin
25         state <= A;
26     end
27
28     else if (Modo == 1'b0 && state == R1)begin
29         state <= nextstate;
30     end
31
32     else begin
33         state <= nextstate;
34     end
35 end
36
37 //nextstate logic
38 always @(*) begin
39     case(state)
40         A: nextstate = R1;
41         R1: nextstate = R2;
42         R2: nextstate = R3;
43         R3: nextstate = R4;
44         R4: nextstate = V1;
45         V1: nextstate = V2;
46         V2: nextstate = V3;
47         V3: nextstate = A;
```

```
48     default: nextstate = A;
49     endcase
50 end
51
52 always @(*) begin
53     if (state == A) begin
54         AS_out = 1'b1;
55     end
56
57     else begin
58         AS_out = 1'b0;
59     end
60 end
61
62 assign A_out = AS_out;
63
64 always @(*) begin
65     if (state == R1 || state == R2 || state == R3 || state == R4) begin
66         RS_out = 1'b1;
67     end
68
69     else begin
70         RS_out = 1'b0;
71     end
72 end
73
74 assign R_out = RS_out;
75
76 always @(*) begin
77     if (state == V1 || state == V2 || state == V3) begin
78         VS_out = 1'b1;
79     end
80
81     else begin
82         VS_out = 1'b0;
83     end
84 end
85
86 assign V_out = VS_out;
87
88 endmodule
89
```

Joshua Alexis Máynez Alcayde
243990

Testbench

```
3 //Verilog HDL for "Verilog", "FSM_tb" "functional"
4
5 //FSM TEST BENCH
6
7 module FSM_tb ();
8
9 reg Clk;
10 reg Modo;
11 wire V_out;
12 wire A_out;
13 wire R_out;
14
15 FSM uut (V_out, A_out, R_out, Clk, Modo);
16
17 initial begin
18
19     $display("Start");
20     Modo = 1;
21     Clk = 0;
22     #10
23     Clk = 1;
24     #10
25     Clk = 0;
26     #10
27     Clk = 1;
28     #10
29     Clk = 0;
30
31     #10;
32     //Llego a R1
33     Clk = 1;
34     #10
35     Clk = 0;
36     #10
37     //Regresa a A
38     Clk = 1;
39     #10
40     Clk = 0;
41     #10
42     //Sigue en A
43     Clk = 1;
44     #10
45     Clk = 0;
46     #10
47     Modo = 0;
48     //---
```

```
48 //R1
49 Clk = 1;
50 #10
51 Clk = 0;
52 #10
53 //R2
54 Clk = 1;
55 #10
56 Clk = 0;
57 #10
58 //R3
59 Clk = 1;
60 #10
61 Clk = 0;
62 #10
63 //R4
64 Clk = 1;
65 Modo = 1;
66 #10
67 Clk = 0;
68 #10
69 //V1
70 Clk = 1;
71 #10
72 Clk = 0;
73 #10
74 //V2
75 Clk = 1;
76 #10
77 Clk = 0;
78 #10
79 //V3
80 Clk = 1;
81 #10
82 Clk = 0;
83 #10
84 //A
85 Clk = 1;
86 #10
87 Clk = 0;
88 #10
89 Clk = 1;
90 #10
91 Clk = 0;
92 #10
93 Clk = 1;
94 #10
```

Joshua Alexis Máynez Alcayde
243990

```
95     Clk = 0;  
96     #10  
97     Clk = 1;  
98     #10  
99     Clk = 0;  
100    #10  
101    Clk = 1;  
102    #10  
103    Clk = 0;  
104    #10  
105        $display("Test completed");  
106  
107 end  
108  
109 endmodule  
110
```

Simulation



Area report

```
@genus:root: 7> report area  
=====
```

Generated by:	Genus(TM) Synthesis Solution 21.18-s082_1
Generated on:	Oct 25 2024 02:19:19 am
Module:	FSM
Technology library:	gscl45nm
Operating conditions:	typical (balanced_tree)
Wireload mode:	enclosed
Area mode:	timing library

```
=====
```

Instance	Module	Cell Count	Cell Area	Net Area	Total Area	Wireload
FSM		18	57.724	0.000	57.724	<none> (D)

(D) = wireload is default in technology library

```
@genus:root: 8>
```

Gates report

```
@genus:root: 8> report gates
=====
Generated by:      Genus(TM) Synthesis Solution 21.18-s082_1
Generated on:      Oct 25 2024  02:20:02 am
Module:           FSM
Technology library: gsc145nm
Operating conditions: typical (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

  Gate      Instances   Area   Library
-----
AND2X1         1    2.346   gsc145nm
BUF2X1         3    7.039   gsc145nm
DFFPOSX1       3   23.934   gsc145nm
INVX1          4    5.632   gsc145nm
NAND3X1        2    4.693   gsc145nm
NOR2X1         2    4.693   gsc145nm
OR2X1          2    4.693   gsc145nm
XNOR2X1        1    4.693   gsc145nm
-----
total          18   57.724

  Type      Instances   Area   Area %
-----
sequential         3   23.934   41.5
inverter           4    5.632    9.8
buffer            3    7.039   12.2
logic             8   21.118   36.6
physical_cells     0    0.000    0.0
-----
total             18   57.724  100.0

@genus:root: 9> █
```

Power report

```
@genus:root: 9> report power
Info    : Joules engine is used. [RPT-16]
Info    : Joules engine is being used for the command report power.
Info    : ACTP-0001 [ACTPInfo] Activity propagation started for stim#0 netlist
Info    : FSM
Info    : ACTP-0009 [ACTPInfo] Activity Propagation Progress Report : 100%
Info    : ACTP-0001 Activity propagation ended for stim#0
Info    : PWRA-0001 [PwrInfo] compute_power effective options
Info    : -mode : vectorless
Info    : -skip_propagation : 1
Info    : -frequency_scaling_factor : 1.0
Info    : -use_clock_freq : stim
Info    : -stim :/stim#0
Info    : -fromGenus : 1
Info    : ACTP-0001 Timing initialization started
Info    : ACTP-0001 Timing initialization ended
Info    : PWRA-0002 [PwrInfo] Skipping activity propagation due to -skip_ap
Info    : option....
Warning: PWRA-0302 [PwrWarn] Frequency scaling is not applicable for vectorless
Warning: PWRA-0304 [PwrWarn] -stim option is not applicable with vectorless mode
Info    : PWRA-0002 Started 'vectorless' power computation.
Info    : PWRA-0002 Finished power computation.
Info    : PWRA-0007 [PwrInfo] Completed successfully.
Info    : Info=6, Warn=2, Error=0, Fatal=0
Instance: /FSM
Power Unit: W
PDB Frames: /stim#0/frame#0
-----
Category      Leakage      Internal      Switching      Total      Row%
-----
memory        0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
register      1.64932e-07  8.70363e-07  4.22290e-07  1.45759e-06  61.31%
latch         0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
logic         1.56314e-07  4.78962e-07  2.84471e-07  9.19747e-07  38.69%
bbox          0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
clock         0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
pad           0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
pm            0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00  0.00%
-----
Subtotal      3.21246e-07  1.34933e-06  7.06761e-07  2.37733e-06  100.00%
Percentage    13.51%      56.76%      29.73%      100.00%  100.00%
-----
@genus:root: 10> █
```


Joshua Alexis Máynez Alcayde
243990

Timing report

```
@genus:root: 11> report timing -unconstrained
=====
Generated by:      Genus(TM) Synthesis Solution 21.18-s082_1
Generated on:      Oct 25 2024  02:21:41 am
Module:            FSM
Operating conditions:  typical (balanced_tree)
Wireload mode:     enclosed
Area mode:         timing library
=====

Path 1: UNCONSTRAINED Setup Check with Pin state_reg[1]/CLK->D
Startpoint: (R) state_reg[1]/CLK
Endpoint: (F) state_reg[1]/D

      Setup:-      57
Data Path:-      242

#-----#
# Timing Point   Flags   Arc   Edge   Cell       Fanout Load Trans Delay Arrival Instance
#              (fF)   (ps)   (ps)   (ps)   (ps)   (ps)   (ps)   Location
#-----#
state_reg[1]/CLK -      -      R      (arrival)   3      -      0      0      0      (-,-)
state_reg[1]/Q   -      CLK->Q F      DFFPOSX1    2  3.7    21    104    104    (-,-)
g304__3680/Y     -      B->Y   F      OR2X1       3  7.0    26    59    164    (-,-)
g297__5107/Y     -      C->Y   R      NAND3X1     1  1.5    33    32    195    (-,-)
drc_bufs321/Y    -      A->Y   R      BUFEX2      1  1.6     8    34    229    (-,-)
g296/Y          -      A->Y   F      INVX1       1  1.3     9    13    242    (-,-)
state_reg[1]/D   -      -      F      DFFPOSX1    1      -      -      0    242    (-,-)
#-----#

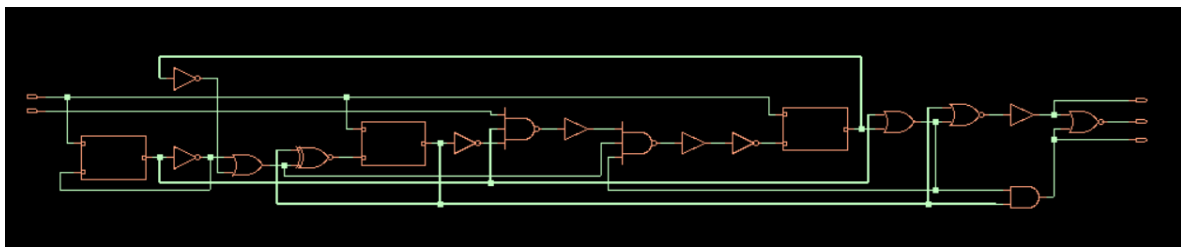
@genus:root: 12> █
```

```
// Generated by Cadence Genus(TM) Synthesis Solution 21.18-s082_1
// Generated on: Oct 25 2024 02:22:36 CST (Oct 25 2024 08:22:36 UTC)

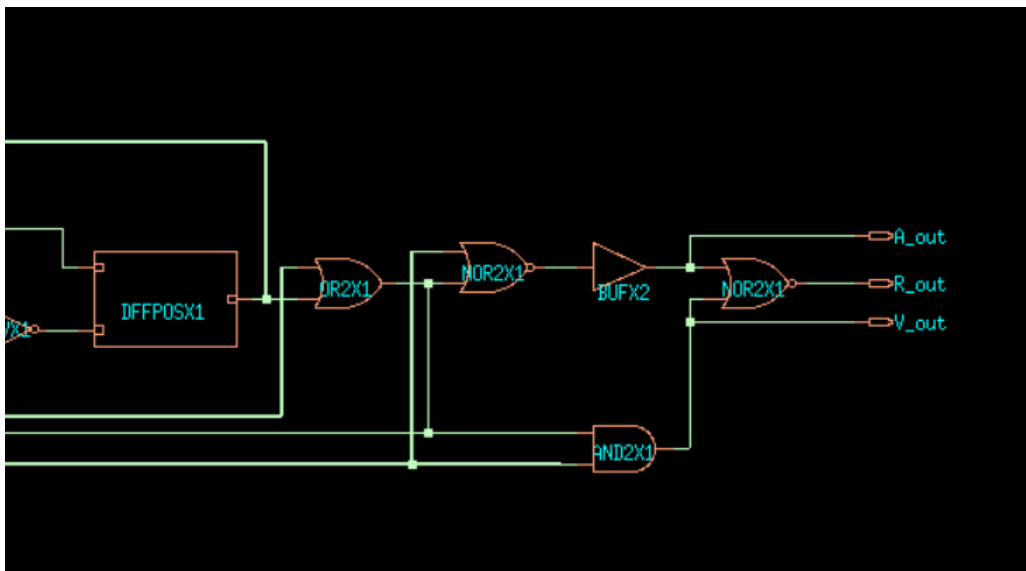
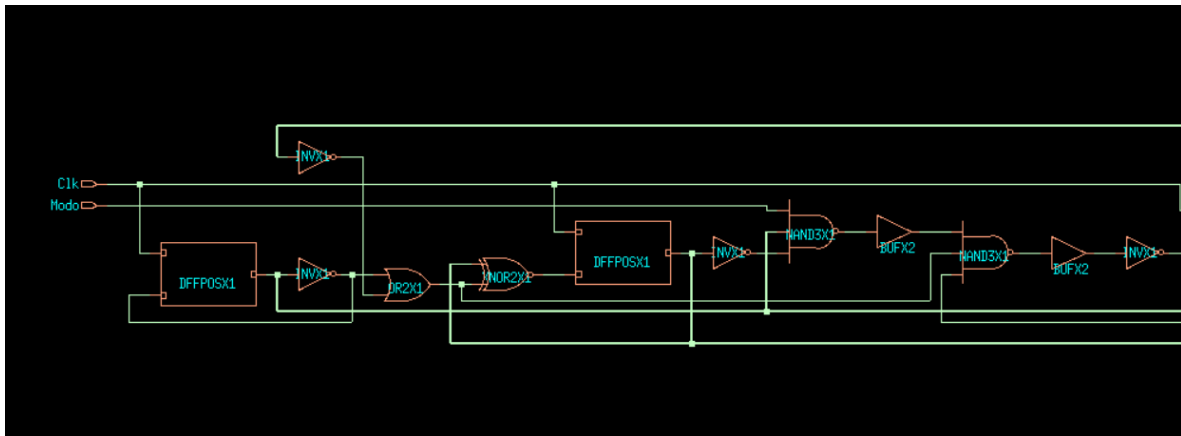
// Verification Directory fv/FSM

module FSM(V_out, A_out, R_out, Clk, Modo);
  input Clk, Modo;
  output V_out, A_out, R_out;
  wire Clk, Modo;
  wire V_out, A_out, R_out;
  wire [2:0] state;
  wire n_0, n_1, n_2, n_3, n_4, n_6, n_8, n_9;
  wire n_10, n_11, n_15, n_28;
  DFFPOSX1 \state_reg[1] (.CLK (Clk), .D (n_11), .Q (state[1]));
  DFFPOSX1 \state_reg[2] (.CLK (Clk), .D (n_10), .Q (state[2]));
  INVX1 g296(.A (n_28), .Y (n_11));
  XNOR2X1 g298__2398(.A (state[2]), .B (n_8), .Y (n_10));
  NAND3X1 g297__5107(.A (n_15), .B (n_8), .C (n_4), .Y (n_9));
  NOR2X1 g295__6260(.A (A_out), .B (V_out), .Y (R_out));
  AND2X1 g299__4319(.A (n_4), .B (state[2]), .Y (V_out));
  NAND3X1 g300__8428(.A (Modo), .B (state[0]), .C (n_0), .Y (n_3));
  NOR2X1 g301__5526(.A (state[2]), .B (n_4), .Y (n_6));
  OR2X1 g303__6783(.A (n_2), .B (n_1), .Y (n_8));
  DFFPOSX1 \state_reg[0] (.CLK (Clk), .D (n_2), .Q (state[0]));
  OR2X1 g304__3680(.A (state[0]), .B (state[1]), .Y (n_4));
  INVX1 g305(.A (state[1]), .Y (n_1));
  INVX1 g307(.A (state[2]), .Y (n_0));
  INVX1 g306(.A (state[0]), .Y (n_2));
  BUFX2 drc_bufs(.A (n_3), .Y (n_15));
  BUFX2 drc_bufs321(.A (n_9), .Y (n_28));
  BUFX2 drc_bufs324(.A (n_6), .Y (A_out));
endmodule
```

Schematic



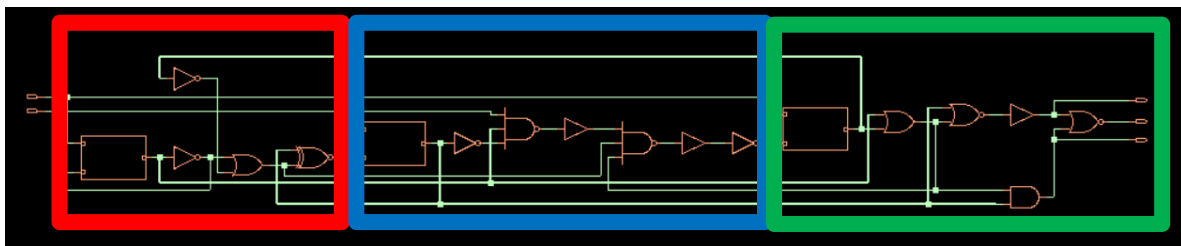
Schematic zoomed in



How many flip-flops are used?

There are 3 flipflops, I believe there is one flipflop for each part of the state machines that are the memory, next state and output.

This are the three basic parts of a state machine.



Red = Memory

Blue = Next state

Green =Output

Joshua Alexis Máynez Alcayde
243990

How much percentage is made with flip-flops and how many with combinational logic?

We only have the 3 flip-flops that I already mentioned and the rest of the circuit its combinational the implementation that genus made only uses the 3 flip-flops and 15 more gates that helps the combinational logic.

Joshua Alexis Máynez Alcayde
243990

CODES

```
//Verilog HDL for "Verilog", "FSM" "functional"  
module FSM (V_out, A_out, R_out, Clk, Modo);
```

```
input Clk;  
input Modo;  
output V_out, A_out, R_out;  
reg VS_out = 0;  
reg AS_out = 1;  
reg RS_out = 0;  
reg [2:0] state = 0;  
reg [2:0] nextstate = 3'b001;
```

```
localparam A = 3'b000;  
localparam R1 = 3'b001;  
localparam R2 = 3'b010;  
localparam R3 = 3'b011;  
localparam R4 = 3'b100;  
localparam V1 = 3'b101;  
localparam V2 = 3'b110;  
localparam V3 = 3'b111;
```

```
//state register  
always @(posedge Clk) begin  
    if (Modo == 1'b1 && state == R1) begin  
        state <= A;  
    end  
  
    else if (Modo == 1'b0 && state == R1) begin  
        state <= nextstate;  
    end  
  
    else begin  
        state <= nextstate;  
    end  
end
```

```
//nextstate logic  
always @(*) begin  
    case(state)  
        A: nextstate = R1;  
        R1: nextstate = R2;  
        R2: nextstate = R3;
```

Joshua Alexis Máynez Alcayde
243990

```
    R3: nextstate = R4;
    R4: nextstate = V1;
    V1: nextstate = V2;
    V2: nextstate = V3;
    V3: nextstate = A;
    default: nextstate = A;
    endcase
end

always @(*) begin
    if (state == A) begin
        AS_out = 1'b1;
    end

    else begin
        AS_out = 1'b0;
    end
end

assign A_out = AS_out;

always @(*) begin
    if (state == R1 || state == R2 || state == R3 || state == R4) begin
        RS_out = 1'b1;
    end

    else begin
        RS_out = 1'b0;
    end
end

assign R_out = RS_out;

always @(*) begin
    if (state == V1 || state == V2 || state == V3) begin
        VS_out = 1'b1;
    end

    else begin
        VS_out = 1'b0;
    end
end

assign V_out = VS_out;
```

Joshua Alexis Máynez Alcayde
243990

```
endmodule
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
//Verilog HDL for "Verilog", "FSM_tb" "functional"

//FSM TEST BENCH

module FSM_tb ();

reg Clk;
reg Modo;
wire V_out;
wire A_out;
wire R_out;

FSM uut (V_out, A_out, R_out, Clk, Modo);

initial begin

    $display("Start");
    Modo = 1;
        Clk = 0;
        #10
        Clk = 1;
        #10
        Clk = 0;
        #10
        Clk = 1;
        #10
        Clk = 0;

    #10;
    //Llego a R1
    Clk = 1;
    #10
    Clk = 0;
    #10
    //Regresa a A
    Clk = 1;
    #10
    Clk = 0;
    #10
    //Sigue en A
```

Joshua Alexis Máynez Alcayde
243990

```
Clk = 1;
#10
Clk = 0;
#10
Modo = 0;
//R1
Clk = 1;
#10
Clk = 0;
#10
//R2
Clk = 1;
#10
Clk = 0;
#10
//R3
Clk = 1;
#10
Clk = 0;
#10
//R4
Clk = 1;
Modo = 1;
#10
Clk = 0;
#10
//V1
Clk = 1;
#10
Clk = 0;
#10
//V2
Clk = 1;
#10
Clk = 0;
#10
//V3
Clk = 1;
#10
Clk = 0;
#10
//A
Clk = 1;
#10
```


Joshua Alexis Máynez Alcayde
243990

```
    Clk = 0;
    #10
    Clk = 1;
    #10
    Clk = 0;
    #10
    Clk = 1;
    #10
    Clk = 0;
    #10
    Clk = 1;
    #10
    Clk = 0;
    #10
    Clk = 1;
    #10
    Clk = 0;
    #10
    $display("Test completed");

end

endmodule
```